

CODTECH IT SOLUTIONS PVT. LTD.

Information Technology Services

Internship Project Report

Sales Trend Visualization

Submitted by

Anushka Jain

Intern ID: CITS3178

Domain: Data Analytics

Internship Period: 03 June 2026 – 15 July 2026

Submitted to

Neela Santhosh Kumar

Human Resources & Academic Head

Project Outputs

Output 1: KNN Classifier – High Sales Prediction

```
user_input = pd.DataFrame()

user_input['Quantity'] = [int(input("Order Quantity: "))]
user_input['Discount'] = [float(input("Discount (as decimal, e.g., 0.1 for 10%): "))]
user_input['Year'] = [int(input("Year (2011-2014): "))]
user_input['Month'] = [int(input("Month (1-12): "))]

user_input_scaled = scaler1.transform(user_input)

prediction = KNN.predict(user_input_scaled)[0]

print("\nPredicted High Sales Classification (0 = Low, 1 = High):", prediction)
print("This means:", label_high_sales(prediction))

*** Order Quantity: 50
Discount (as decimal, e.g., 0.1 for 10%): 0.10
Year (2011-2014): 2012
Month (1-12): 6

Predicted High Sales Classification (0 = Low, 1 = High): 1
This means: High Sales Transaction
```

Fig. 1: KNN model predicts whether a transaction is High or Low Sales based on quantity, discount, year, and month. Input: Qty=50, Discount=10%, Year=2012, Month=6 → Predicted: High Sales Transaction.

Output 2: Decision Tree Classifier – Sample Run 1

```
clf2 = DecisionTreeClassifier(random_state=42)
clf2.fit(x3_train, y3_train)

print("Please enter the following transaction information: \n")

test = pd.DataFrame({
    'Quantity': [int(input('Enter Order Quantity: '))],
    'Discount': [float(input('Enter Discount Rate (0-1, e.g., 0.1 for 10%): '))],
    'Year': [int(input('Enter Year (2011-2014): '))],
    'Month': [int(input('Enter Month (1-12): '))],
    'Region': [input('Enter Region (East/West/South/Central): ')],
    'Category': [input('Enter Category (Technology/Furniture/Office Supplies): ')],
    'Segment': [input('Enter Segment (Consumer/Corporate/Home Office): ')],
    'Ship Mode': [input('Enter Ship Mode (Standard/Second Class/First Class/Same Day): ')]
})

test_encoded = pd.get_dummies(test)
test_encoded = test_encoded.reindex(columns=x3.columns, fill_value=0)
prediction = clf2.predict(test_encoded)

result = "Yes, High Sales Transaction" if prediction[0] == 1 else "No, Low Sales Transaction"

print("\nPredicted as High Sales: ", result)
```

```
... Please enter the following transaction information:

Enter Order Quantity: 500
Enter Discount Rate (0-1, e.g., 0.1 for 10%): 0
Enter Year (2011-2014): 2014
Enter Month (1-12): 12
Enter Region (East/West/South/Central): West
Enter Category (Technology/Furniture/Office Supplies): Technology
Enter Segment (Consumer/Corporate/Home Office): Corporate
Enter Ship Mode (Standard/Second Class/First Class/Same Day): First Class

Predicted as High Sales: No, Low Sales Transaction
```

Fig. 2: Decision Tree model predicts sales category using full transaction details. Input: Qty=500, Discount=0, Year=2014, Month=12, Region=West, Category=Technology, Segment=Corporate, Ship Mode=First Class → Predicted: No, Low Sales Transaction.

Output 3: Decision Tree Classifier – Sample Run 2

```
clf2 = DecisionTreeClassifier(random_state=42)
clf2.fit(x3_train, y3_train)

print("Please enter the following transaction information: \n")

test = pd.DataFrame({
    'Quantity': [int(input('Enter Order Quantity: '))],
    'Discount': [float(input('Enter Discount Rate (0-1, e.g., 0.1 for 10%): '))],
    'Year': [int(input('Enter Year (2011-2014): '))],
    'Month': [int(input('Enter Month (1-12): '))],
    'Region': [input('Enter Region (East/West/South/Central): ')],
    'Category': [input('Enter Category (Technology/Furniture/Office Supplies): ')],
    'Segment': [input('Enter Segment (Consumer/Corporate/Home Office): ')],
    'Ship Mode': [input('Enter Ship Mode (Standard/Second Class/First Class/Same Day): ')]
})

test_encoded = pd.get_dummies(test)
test_encoded = test_encoded.reindex(columns=x3.columns, fill_value=0)
prediction = clf2.predict(test_encoded)

result = "Yes, High Sales Transaction" if prediction[0] == 1 else "No, Low Sales Transaction"

print("\nPredicted as High Sales: ", result)
```

```
... Please enter the following transaction information:

Enter Order Quantity: 20
Enter Discount Rate (0-1, e.g., 0.1 for 10%): 0.4
Enter Year (2011-2014): 2011
Enter Month (1-12): 3
Enter Region (East/West/South/Central): Central
Enter Category (Technology/Furniture/Office Supplies): Office Supplies
Enter Segment (Consumer/Corporate/Home Office): Home Office
Enter Ship Mode (Standard/Second Class/First Class/Same Day): Standard Class

Predicted as High Sales: No, Low Sales Transaction
```

Fig. 3: Second test run of the Decision Tree Classifier. Input: Qty=20, Discount=40%, Year=2011, Month=3, Region=Central, Category=Office Supplies, Segment=Home Office, Ship Mode=Standard Class → Predicted: No, Low Sales Transaction.

Output 4: Logistic Regression – High Sales Probability

```
user_input2 = pd.DataFrame()
user_input2['Profit'] = [float(input("Profit: "))]
user_input2['Quantity'] = [int(input("Order Quantity: "))]
user_input2['Discount'] = [float(input("Discount Rate (0-1): "))]
user_input2['Year'] = [int(input("Year: "))]
user_input2['Month'] = [int(input("Month: "))]
user_input2['Sales'] = [float(input("Sales: "))]

user_input2_scaled = scaler2.transform(user_input2)
prediction = log_model.predict(user_input2_scaled)[0]
probability = log_model.predict_proba(user_input2_scaled)[0][1]

if prediction == 1:
    print("\nPrediction: YES – This transaction is likely to be a High Sales transaction.")
else:
    print("\nPrediction: NO – This transaction is unlikely to be a High Sales transaction.")

print(f"Estimated High Sales Probability: {probability:.2%}")

... Profit: 500
Order Quantity: 300
Discount Rate (0-1): 0.05
Year: 2014
Month: 1
Sales: 100

Prediction: YES – This transaction is likely to be a High Sales transaction.
Estimated High Sales Probability: 100.00%
```

Fig. 4: Logistic Regression model predicts the probability of a High Sales transaction. Input: Profit=500, Qty=300, Discount=5%, Year=2014, Month=1, Sales=100 → Prediction: YES (High Sales), Estimated Probability: 100.00%.

Output 5: Linear Regression – Predicted Sales Amount

```

# Scale the input using the same scaler
user_regression_scaled = scaler_reg.transform(user_regression_input)

# Make prediction
predicted_sales = linear_reg_model.predict(user_regression_scaled)[0]

# Display result
print("\n" + "=" * 60)
print("PREDICTION RESULT")
print("=" * 60)
print(f"Input Transaction Details:")
print(f"  - Quantity: {user_quantity} units")
print(f"  - Discount: {user_discount*100:.1f}%")
print(f"  - Year: {user_year}")
print(f"  - Month: {user_month}")
print(f"\n✓ PREDICTED SALES AMOUNT: ${predicted_sales:.2f}")
print("=" * 60)

```

```

=====
PREDICT FUTURE SALES AMOUNT
=====

Enter transaction details to predict the sales dollar amount:

Order Quantity (units): 100
Discount Rate (0-1, e.g., 0.1 for 10%): 0.10
Year (2011-2014): 2013
Month (1-12): 6

=====
PREDICTION RESULT
=====

Input Transaction Details:
  - Quantity: 100.0 units
  - Discount: 10.0%
  - Year: 2013
  - Month: 6

✓ PREDICTED SALES AMOUNT: $5307.96
=====

```

Fig. 5: Linear Regression model forecasts the dollar value of a future sale. Input: Qty=100 units, Discount=10%, Year=2013, Month=6 → Predicted Sales Amount: \$5307.96.